

Website Traffic Classification

Matthew Harris

mjharris02@wm.edu

William & Mary

Williamsburg, Virginia, USA

Abstract

Classifying the origin website of encrypted network traffic is a practical privacy and security problem with applications in traffic analysis, network management, and censorship research. In this project I build a supervised classifier that predicts whether a packet capture belongs to YouTube, Wikipedia, or Agar.io using only metadata derived from packet headers and timing, without using plaintext payloads or URLs.

I construct a labeled dataset from repeated visits to each website and extract flow level and burst oriented statistics. I compare simple baselines (linear and logistic regression) against a gradient boosted decision tree model (LightGBM[1]) trained on an engineered feature set. Using grouped cross validation by capture file to reduce leakage, my LightGBM model achieves mean accuracy of 0.821 and macro F1 of 0.792. A lightweight hyperparameter sweep improves performance to 0.831 accuracy and 0.802 macro F1 on the same evaluation protocol. I also study the effect of the flow timeout used during feature extraction and show that timeout choice measurably impacts downstream accuracy.

Keywords

traffic analysis, encrypted traffic classification, flow features, gradient boosting, LightGBM

1 Introduction

Transport encryption has made network payload inspection impractical for many classification tasks, but packet metadata remains highly informative. Even when application data is encrypted, traffic traces can leak information through timing patterns, packet sizes, flow counts, and burst structure. This project targets a constrained but realistic setting: given a packet capture (PCAP/PCAPNG) from a browsing session, predict which website was visited among three candidate websites. Packet capture formats and analysis tooling such as Wireshark and tshark are used to create and inspect traces during dataset construction[3].

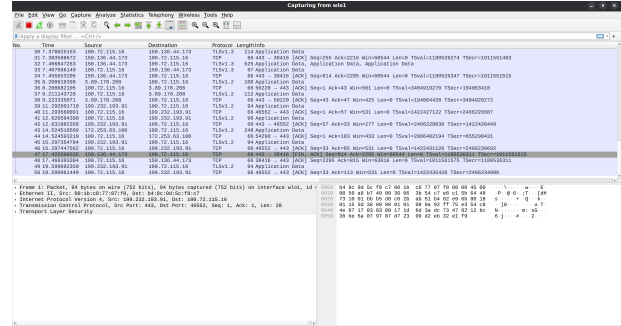


Figure 1: Screenshot of Wireshark/tshark used to inspect packet captures during dataset construction.

The three websites used in this work are YouTube, Wikipedia, and Agar.io. They represent distinct interaction patterns: YouTube tends to generate sustained high volume downloads, Wikipedia tends to generate many short request response bursts, and Agar.io is interactive with frequent lower volume updates.

While the setting is simplified relative to open world fingerprinting, it captures key challenges that generalize: the need to build robust flow based features, to avoid evaluation leakage across repeated visits, and to understand feature sensitivity to preprocessing choices such as flow timeout.

This includes:

- A reproducible dataset construction pipeline from raw PCAPNG files into a tabular dataset.
- A feature engineering approach that combines classic flow statistics with burst related features designed to better capture website specific structure.
- A leakage resistant evaluation protocol using grouped cross validation by capture file.
- An empirical study of the impact of flow timeout on classification performance.

2 Proposed Method

This section describes the dataset, the feature extraction pipeline, the preprocessing and modeling approach, and how I avoid common evaluation pitfalls.

2.1 Dataset and label space

Our dataset consists of packet captures collected from repeated visits to YouTube, Wikipedia, and Agar.io. Each capture file is treated as a single labeled example. Captures were performed on the same Linux machine to improve repeatability, with background activity minimized, and multiple users were asked to interact with each service in a natural way (watching videos, reading articles, and playing the game).

Captures were collected in fixed intervals of approximately five minutes to produce multiple PCAPNG files per label.

I restrict inputs to header and timing metadata only and do not use payload content.

Each example is associated with a `source_file` identifier derived from the capture filename. This identifier is critical for evaluation because multiple rows can be produced from the same capture due to feature extraction choices. I use it as a grouping key in cross validation to prevent the same capture from appearing in both training and validation folds[2].

Figure 2: Overview of the dataset used for traffic classification.

2.2 Flow reconstruction and flow timeout

Raw packets are converted into bidirectional flows using the standard 5 tuple (source IP, destination IP, source port, destination port, transport protocol). Packets in both directions are grouped into the same flow. A flow timeout T seconds determines when inactivity closes a flow and a new flow entry begins.

Timeout is a classic parameter in NetFlow style aggregation. If T is too small, long lived connections can be fragmented into many flows, inflating flow counts and diluting statistics. If T is too large, behaviors that occur as distinct phases may be merged, smoothing informative differences. I therefore evaluate a family of datasets built with different timeouts and quantify performance sensitivity in Section 3.5.

2.3 Feature extraction

For each capture I compute a feature vector composed of:

- (1) **Flow summary statistics:** counts, durations, total bytes, packet counts, and directionality aggregated across flows.

- (2) **Heavy tail features:** statistics that emphasize large flows or large bursts to capture skewed distributions.
- (3) **Burst features:** aggregated measures of contiguous sequences of packets in the same direction, intended to capture download and upload phases common in web browsing.

I also compute class specific permutation importances on the trained model to understand which engineered features are most predictive. The top features by class are visualized in Figure 6.

Here are the top features:

Feature	Purpose
flow_duration	captures long-lived / constant connections
iat_mean / iat_std	inter-packet timing; burstiness vs steady flow
burst_count_t20 / avg_burst_bytes_t20	short-timescale spikes — identifies bursts
byte_ratio_up_down	directionality (download vs upload-heavy)

Figure 3: Summary of engineered feature families used to represent each capture.

2.4 Models

These models were compared:

- **Linear baseline:** a simple linear classifier trained on standardized features.
- **Logistic regression:** multinomial logistic regression as a stronger linear baseline.
- **LightGBM:** gradient boosted decision trees (GBDT) trained on the engineered feature set.

LightGBM is a strong choice for tabular features with nonlinear interactions and mixed scales. It also provides feature importance tooling that supports my analysis.

2.4.1 LightGBM: definition and rationale. LightGBM is an implementation of gradient boosted decision trees that builds an ensemble of shallow trees in a stagewise fashion. Each tree corrects residual errors from the previous ensemble, and the final prediction is a weighted sum of tree outputs. Compared to bagging approaches such as Random Forest, boosted trees focus model capacity on the hard examples and model residual structure directly.

There are several reasons LightGBM is a good fit for this project. First, flow and burst features are heterogeneous and often interact in nonlinear ways; boosted trees naturally capture those interactions without feature scaling. Second, LightGBM is computationally efficient and supports sparsity and categorical handling, which makes it easy to iterate on

feature sets. Third, the model exposes clear feature importance measures such as permutation importance which we use to interpret per class signals.

In my experiments LightGBM slightly outperformed Random Forest. Likely causes are that LightGBM’s boosting objective concentrates learning on difficult decision boundaries such as the Agar.io border cases, and its leaf-wise tree growth can produce more expressive combinations of features for a given model size. Random Forest produces decorrelated trees by bagging but does not iteratively focus on misclassified examples, which can make it less effective when a relatively small number of feature interactions determine hard class boundaries.

2.5 Preprocessing and leakage resistant validation

I standardize numeric features (when required by the model) and ensure the same feature set is used across folds. Because repeated captures can share low level artifacts, I use grouped cross validation: all rows derived from a given `source_file` belong to the same fold. This better approximates performance on unseen captures.

To mitigate moderate class imbalance, oversampling can be performed within each training fold only. In my best performing configuration, LightGBM already handled imbalance reasonably well, and I primarily report the grouped cross validation results that match the dataset and feature pipeline.

3 Evaluation

I evaluate models on grouped cross validation using accuracy and macro F1. Accuracy captures overall correctness, while macro F1 averages per class performance and is sensitive to minority class performance.

3.1 Experimental setup

All results in this section are reported using k fold grouped cross validation over `source_file`. I compute mean metrics across folds.

The primary figures referenced in this paper are produced from the project repository and are included directly from the directory `baseline_results/presentation_selected/`.

3.2 Model comparisons

Figure 2 provides a compact comparison of baseline models with the engineered feature pipeline. On my grouped cross validation protocol, LightGBM significantly outperforms linear and logistic regression baselines.

Quantitatively, the LightGBM model with engineered features achieves mean accuracy 0.821 and macro F1 0.792, while

the linear baseline achieves 0.696 accuracy and 0.658 macro F1.

Model comparison — CV mean (Δ vs LightGBM)

Model	Accuracy	StdAcc	MacroF1	StdF1	Δ Acc vs LGBM	Δ F1 vs LGBM
Ridge (linear)	0.696	0.046	0.658	0.052	-0.124	-0.133
Logistic	0.661	0.042	0.627	0.048	-0.160	-0.164
LightGBM (best)	0.821	0.024	0.792	0.021	+0.000	+0.000

Figure 4: Grouped cross validation model comparison (three class classification). Values shown are aggregated across folds.

3.3 Hyperparameter tuning

I ran a small hyperparameter sweep over key LightGBM parameters (number of estimators, learning rate, tree depth and leaf constraints, and subsampling). Figure 3 visualizes the baseline and tuned comparisons.

The best tuned configuration improves mean performance from 0.821 to 0.831 accuracy and from 0.792 to 0.802 macro F1.

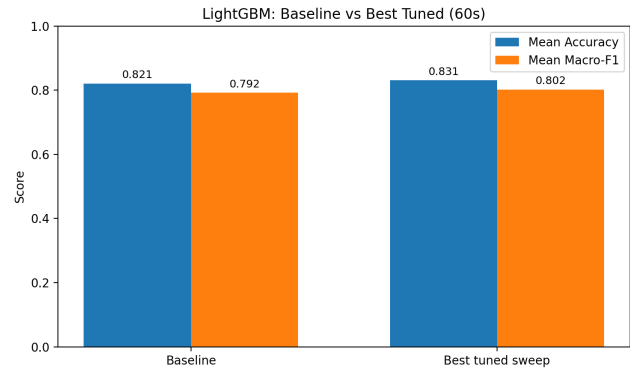


Figure 5: Baseline versus tuned LightGBM performance on the same grouped cross validation protocol.

3.4 Confusion matrix analysis

To interpret failure modes, I aggregate predictions across folds and compute an overall confusion matrix (Figure 4). Confusions often occur between websites that share similar content delivery patterns or similar burst structure.

In my experiments, Agar.io is the label that is most often misclassified. The aggregate confusion matrix shows that, of 163 true Agar.io examples, 119 are correctly predicted as Agar.io (recall 73.1%) while 26 are predicted as YouTube and 18 as Wikipedia (27.0% total misclassification; 16.0% -> YouTube, 11.0% -> Wikipedia). By contrast, YouTube examples are correctly predicted 464 of 529 times (87.7% recall)

and Wikipedia 224 of 292 times (76.7% recall). These numbers confirm that Agar.io sits near the decision boundary: its traffic sometimes resembles YouTube (persistent connections, occasional streamed media) and sometimes resembles Wikipedia (short bursts and lower sustained volume), which increases its rate of being confused with the two other labels.

Possible mitigations include adding features that better capture very short-timescale interactivity (for example high resolution burst variance or sub-second peak rates), collecting more diverse Agar.io sessions (including with and without ad content), and using a multi-stage classifier that first separates sustained streaming from interactive sessions and then classifies within the chosen regime.

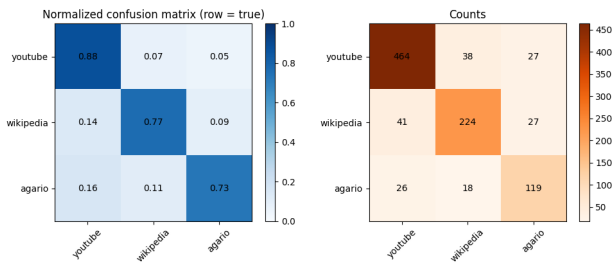


Figure 6: Aggregate confusion matrix of the best performing LightGBM model under grouped cross validation.

3.5 Timeout sensitivity

I evaluate extracted datasets built with different flow timeout values to measure sensitivity to preprocessing. Figure 5 summarizes performance across timeouts.

The results show that flow timeout is not a neutral parameter: it changes the shape of derived flows and therefore the engineered features. In practice, timeouts that are too small can fragment long lived connections, while overly large timeouts can merge phases and reduce discriminative power. This tradeoff suggests timeout should be treated as a tunable preprocessing parameter rather than a fixed constant.

In my experiments, a 60 s timeout provided a strong balance between capturing complete interactions and avoiding the merger of unrelated phases, and it served as the primary dataset configuration used for the tuned LightGBM results.

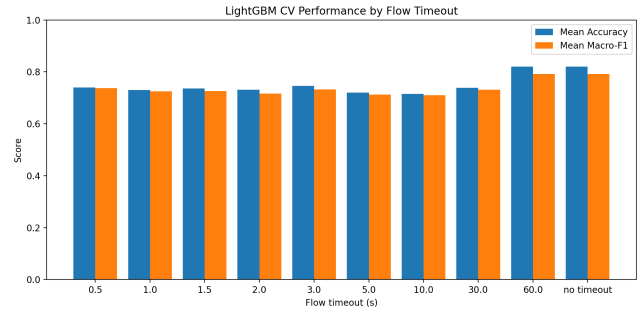


Figure 7: Grouped cross validation performance across multiple flow timeout settings.

3.6 Feature importance

Permutation importance provides an interpretable view into which features drive model predictions. Figure 6 shows the top features for each class.

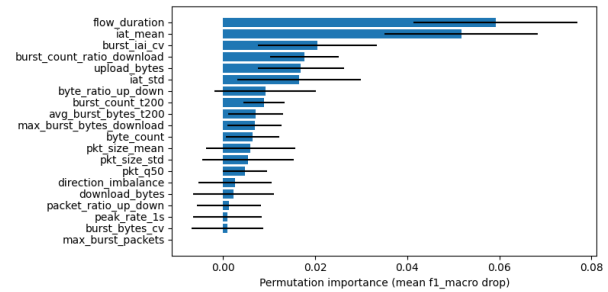


Figure 8: Permutation importance of the top 20 features across all classes.

- `flow_duration`: the total elapsed time from the first packet to the last packet in a flow. Short durations often indicate brief interactive exchanges or failed/quick connections, while long durations indicate sustained downloads or prolonged interactions. In my dataset, Wikipedia flows often show longer durations when a page and its assets are being downloaded, whereas Agar.io flows can be either short (quick joins) or long (extended gameplay). Thus `flow_duration` helps separate long, sustained download-like flows (more likely Wikipedia) from many short or mid-length interactive sessions (often Agar.io), but it is not perfectly discriminative on its own.
- `iat_mean` / `iat_std`: inter-arrival time statistics that summarize packet timing within the flow. `iat_mean` gives the average gap between consecutive packets, while `iat_std` measures timing variability. Low `iat_mean` and low `iat_std` indicate a steady stream of packets, whereas low mean but high std indicates bursts separated by quiet periods. In my experiments, Agar.io tends toward smaller mean

IATs during active gameplay with relatively steady updates, while Wikipedia downloads produce short bursts (low mean) but higher std because the page fetch causes a concentrated burst followed by silence. Together these statistics help the model distinguish rhythmic gameplay traffic from bursty HTTP transfers.

- **burst_count_t20 / avg_burst_bytes_t20:** multi-scale burst features computed using a short window (20 ms) to detect micro-bursts. `burst_count_t20` counts how many such bursts occur in the flow; `avg_burst_bytes_t20` measures the average bytes sent during those bursts. A high burst count with small average bytes suggests many short control or interactive messages; few bursts with large average bytes suggests a small number of heavy transfers. Agar.io often exhibits many small bursts as clients exchange game-state updates frequently, producing higher `burst_count_t20` but lower `avg_burst_bytes_t20`. Wikipedia page fetches often cause fewer but larger bursts as resources are transferred in denser packets.
- **byte_ratio_up_down:** the ratio of bytes sent upstream to bytes sent downstream. It captures asymmetry in client server behavior. Downstream-dominant flows indicate downloads or content delivery, while higher upstream ratios can indicate interactive uploads or signaling traffic. Wikipedia is typically downstream-heavy (low upstream ratio) because clients download pages and assets, while Agar.io gameplay can show higher upstream signalling combined with downstream updates; in practice, Agar.io flows often have a more balanced or higher up/down ratio compared to Wikipedia, making this ratio a useful directional discriminator when combined with timing features.

4 Discussion & Future Work

4.1 Dataset considerations and limitations

This project uses a small closed world label set and a limited number of capture sessions per class. Several factors can inflate apparent performance if not handled carefully:

- **Capture leakage:** if rows derived from the same capture file appear in both train and test splits, models can exploit capture specific artifacts. Grouping by `source_file` mitigates this.
- **Environment coupling:** captures were collected on the same network and device environment. Models could learn environment signatures rather than website signatures.
- **Temporal drift:** website behavior changes over time due to caching, A/B tests, and CDN routing. Periodic re collection is required for long term validity.
- **Ad blocking.** Ad blocking can change the traffic distribution substantially. In my collection environment, DNS level ad blocking reduced the probability that Agar.io would

load large video advertisements that could make its traffic resemble YouTube.

4.2 Modeling considerations

LightGBM performed well with minimal feature scaling and modest tuning, indicating that nonlinear interactions among engineered features are important. Class wise feature importances also suggest that different labels are separable via different subsets of statistics.

Future modeling directions include:

- **Open world evaluation:** adding a rejection option or an “other” class.
- **Robustness:** training on captures collected across multiple devices and networks.
- **Sequence models:** exploring CNN or transformer models over packet size and timing sequences, compared against the tabular approach.
- **Joint optimization:** treating flow timeout, burst definitions, and model hyperparameters as a single optimization space.

4.3 Ethical considerations

Traffic analysis techniques can be used for legitimate security monitoring and performance debugging, but they can also be repurposed for invasive surveillance. My work uses a closed world experimental dataset collected by the authors for educational purposes. I intentionally avoid payload inspection and focus on aggregate metadata. Any deployment of similar methods should include a clear justification, user consent where applicable, and safeguards to prevent misuse.

5 Conclusion

I presented a traffic analysis pipeline that classifies encrypted website traffic using only flow and burst level metadata features. A LightGBM classifier trained on an engineered feature set substantially outperforms linear baselines under grouped cross validation. I also showed that flow timeout selection affects model performance, suggesting that preprocessing parameters should be treated as part of the modeling design. Overall, the project demonstrates that careful feature engineering plus leakage aware evaluation can achieve strong performance in the closed world website classification setting.

The main insights from this study are: 1) engineered flow and burst features provide strong, interpretable signals for distinguishing website traffic when evaluated without leakage; 2) gradient boosted trees (LightGBM) effectively capture nonlinear interactions among these features and benefit from modest tuning; 3) preprocessing choices such as flow timeout materially affect downstream accuracy and should be treated as tunable design decisions; and 4) some interactive

applications (Agar.io) can straddle behavior regimes and require either richer short-timescale features or targeted data collection to reduce misclassification.

References

- [1] Guolin Ke, Qiaozhu Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qi Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*. <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [2] Pedregosa, F. and Varoquaux, G. and Gramfort, A. and Michel, V. and Thirion, B. and Grisel, O. and Blondel, M. and Prettenhofer, P. and Weiss, R. and Dubourg, V. and Vanderplas, J. and Passos, A. and Cournapeau, D. and Brucher, M. and Perrot, M. and Duchesnay, E. 2011. *scikit-learn: Machine Learning in Python*. <https://scikit-learn.org/stable/>.
- [3] The Wireshark Developer Community. 2024. *Wireshark: Network Protocol Analyzer*. <https://www.wireshark.org/>.